`ranges::fold`

Document #:	P2322R5
Date:	2021-10-18
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

Since [P2322R4], removed the short-circuiting fold with one that simply returns the end iterator in addition to the value. Also removed the projections, see the discussion.

LEWG also reconfirmed having `foldl` and `foldl1` under different names, polling "fold with an initial value and fold with no initial value should have the same name (presumably just foldl)" (since once the projections were removed, there is no more ambiguity between the two algorithms)

SF	F	N	A	SA
0	3	4	8	3

LEWG also voted to rename the `*1` suffix to `*_first`. `fold_left` was also preferred to `foldl` (10-6), but there was no consensus between choosing the name `fold` for the simple left fold (as previously preferred) or `fold_left` (for symmetry). This revision has been tentatively updated to use the symmetric names `fold_left`, `fold_left_first`, `fold_right`, and `fold_right_last`, although this will need further discussion.

Since [P2322R3], no changes in actual proposal, just some improvements in the description.

[P2322R2] used the names `fold_left` and `fold_right` to refer to the left- and right- folds and used the same names for the initial value and no-initial value algorithms. LEWG took the following polls [P2322-minutes]:

— `fold` with an initial value, and `fold` with no initial value and non-empty range should have different names (presumably `fold` and `fold_first`)

SF	F	N	A	SA
6	6	3	2	1

— Rename `fold_left` to `fold`

SF	F	N	A	SA
2	10	8	3	1

This revision uses different names for the initial value and no-initial value algorithms, although rather than using `fold` and `fold_right` (and coming up with how to name the no-initial value versions), this paper uses the names `foldl` and `foldr` and then `foldl1` and `foldr1`. This revision also changes the no-initial value versions from having a non-empty range as a precondition to instead returning `optional<T>`.

There was also discussion around having these algorithms return an end iterator.

— Return the end iterator in addition to the result

SF	F	N	A	SA
2	3	5	4	6

— Have a version of the fold algorithms that return the end iterator in addition to the result (as either an additional set of functions, or having some of the versions have different return values)

SF	F	N	A	SA
4	9	3	3	1

But the primary algorithms (`foldl` and `foldl1` for left-fold) will definitely solely return a value. This revision adds further discussion on the flavors of `fold` that can be provided, and ultimately adds another pair that satisfies this desire.

[P2322R1] used *`weakly-assignable-from`* as the constraint, this elevates it to `assignable_from`. This revision also changes the return type of `fold` to no longer be the type of the initial value, see the discussion.

[P2322R0] used `regular_invocable` as the constraint in the *`Foldable`* concept, but we don't need that for this algorithm and it prohibits reasonable uses like a mutating operation. `invocable` is the sufficient constraint here (in the same way that it is for `for_each`). Also restructured the API to overload on providing the initial value instead of having differently named algorithms.

§ 2 Introduction

As described in [P2214R0], there is one very important rangified algorithm missing from the standard library: `fold`.

While we do have an iterator-based version of `fold` in the standard library, it is currently named `accumulate`, defaults to performing `+` on its operands, and is found in the header `<numeric>`. But `fold` is much more than addition, so as described in the linked paper, it's important to give it the more generic name and to avoid a default operator.

Also as described in the linked paper, it is important to avoid over-constraining `fold` in a way that prevents using it for heterogeneous folds. As such, the `fold` specified in this paper only requires one particular invocation of the binary operator and there is no `common_reference` requirement between any of the types involved.

Lastly, the `fold` here is proposed to go into `<algorithm>` rather than `<numeric>` since there is nothing especially numeric about it.

§ 3 Return Type

Consider the example:

```
std::vector<double> v = {0.25, 0.75};
auto r = ranges::fold(v, 1, std::plus());
```

What is the type and value of `r`? There are two choices, which I'll demonstrate with implementations (with incomplete constraints).

§ 3.1 Always return `T`

We implement like so:

```
template <range R, movable T, typename F>
auto fold(R&& r, T init, F op) -> T
{
    ranges::iterator_t<R> first = ranges::begin(r);
    ranges::sentinel_t<R> last = ranges::end(r);
    for (; first != last; ++first) {
        init = invoke(op, move(init), *first);
    }
    return init;
}
```

Here, `fold(v, 1, std::plus())` is an `int` because the initial value is `1`. Since our accumulator is an `int`, the result here is `1`. This is a consistent with `std::accumulate` and is simple to reason about and specify. But it is also a common gotcha with `std::accumulate`.

Note that if we use `assignable_from<T&, invoke_result_t<F&, T, range_reference_t<R>>>` as the constraint on this algorithm, in this example this becomes `assignable_from<int&, double>`. We would be violating the semantic requirements of `assignable_from`, which state 18.4.8 [concept.assignable]/1.5:

(1.5) After evaluating `lhs = rhs`:

(1.5.1) — `lhs` is equal to `rcopy`, unless `rhs` is a non-const xvalue that refers to `lcopy`.

This only holds if all the `double`s happen to be whole numbers, which is not the case for our example. This invocation would be violating the semantic constraints of the algorithm.

§ 3.2 Return the result of the initial invocation

When we talk about the mathematical definition of fold, that's `f(f(f(f(init, x1), x2), ...), xn)`. If we actually evaluate this expression in this context, that's `((1 + 0.25) + 0.75)` which would be `2.0`.

We cannot in general get this type correctly. A hypothetical `f` could actually change its type every time which we cannot possibly implement, so we can't exactly mirror the mathematical definition regardless. But let's just put that case aside as being fairly silly.

We could at least address the gotcha from `std::accumulate` by returning the decayed result of invoking the binary operation with `T` (the initial value) and the reference type of the range. That is, `U = decay_t<invoke_result_t<F&, T, ranges::range_reference_t<R>>>`. There are two possible approaches to implementing a fold that returns `U` instead of `T`:

Two invocation kinds

```
template <range R, movable T, typename
          F,
              typename U = /* ... */>
auto fold(R&& r, T init, F f) -> U
{
    ranges::iterator_t<R> first =
        ranges::begin(r);
    ranges::sentinel_t<R> last =
        ranges::end(r);

    if (first == last) {
        return move(init);
    }

    U accum = invoke(f, move(init),
        *first);
    for (++first; first != last;
        ++first) {
        accum = invoke(f, move(accum),
        *first);
    }
```

One invocation kind

```
template <range R, movable T, typename
          F,
              typename U = /* ... */>
auto fold(R&& r, T init, F f) -> U
{
    ranges::iterator_t<R> first =
        ranges::begin(r);
    ranges::sentinel_t<R> last =
        ranges::end(r);

    U accum = std::move(init);
    for (; first != last; ++first) {
        accum = invoke(f, move(accum),
        *first);
    }
    return accum;
}
```

Two invocation kinds	One invocation kind
`    return accum;` `}`	

Either way, our set of requirements is:

— `invocable<F&, T, range_reference_t<R>>` (even though the implementation on the right does not actually invoke the function using these arguments, we still need this to determine the type `U`)

— `invocable<F&, U, range_reference_t<R>>`

— `convertible_to<T, U>`

— `assignable_from<U&, invoke_result_t<F&, U, range_reference_t<R>>>`

While the left-hand side also needs `convertible_to<invoke_result_t<F&, T, range_reference_t<R>>, U>`.

This is a fairly complicated set of requirements.

But it means that our example, `fold(v, 1, std::plus())` yields the more likely expected result of `2.0`. So this is the version this paper proposes.

§ 4 Other `fold` algorithms

[P2214R0] proposed a single fold algorithm that takes an initial value and a binary operation and performs a *left* fold over the range. But there are a couple variants that are also quite valuable and that we should adopt as a family.

§ 4.1 `fold1`

Sometimes, there is no good choice for the initial value of the fold and you want to use the first element of the range. For instance, if I want to find the smallest string in a range, I can already do that as `ranges::min(r)` but the only way to express this in terms of `fold` is to manually pull out the first element, like so:

```
auto b = ranges::begin(r);
auto e = ranges::end(r);
ranges::fold(ranges::next(b), e, *b, ranges::min);
```

But this is both tedious to write, and subtly wrong for input ranges anyway since if the `next(b)` is evaluated before `*b`, we have a dangling iterator. This comes up enough that this paper proposes a version of `fold` that uses the first element in the range as the initial value (and thus has a precondition that the range is not empty).

This algorithm exists in Scala and Kotlin (which call the non-initializer version `reduce` but the initializer version `fold`), Haskell (under the name `fold1`), and Rust (in the `Itertools` crate under the name `fold1`

and recently finalized under the name `reduce` to match Scala and Kotlin [iterator_fold_self], although at some point it was `fold_first`).

In Python, the single algorithm `functools.reduce` supports both forms (the `initializer` is an optional argument). In Julia, `foldl` and `foldr` both take an optional initial value as well (though it is mandatory in certain cases).

There are two questions to ask about the version of `fold` that does not take an extra initializer.

4.1.1 Distinct name?

Should we give this algorithm a different name (e.g. `fold_first` or `fold1`, since `reduce` is clearly not an option for us) or provide it as an overload of `fold`? To answer that question, we have to deal with the question of ambiguity. For two arguments, `fold(xs, a)` can only be interpreted as a `fold` with no initial value using `a` as the binary operator. For four arguments, `fold(xs, a, b, c)` can only be interpreted as a `fold` with `a` as the initial value, `b` as the binary operation that is the reduction function, and `c` as a unary projection.

What about `fold(xs, a, b)`? It could be:

1. Using `a` as the initial value and `b` as a binary reduction of the form `(A, X) -> A`.
2. Using `a` as a binary reduction of the form `(X, Y) -> X` and `b` as a unary projection of the form `X -> Y`.

Is it possible for these to collide? It would be an uncommon situation, since `b` would have to be both a unary and a binary function. But it is definitely *possible*:

```
inline constexpr auto first = [](auto x, auto... ){ return x; };
auto r = fold(xs, first, first);
```

This call is ambiguous! This works with either interpretation. It would either just return `first` (the lambda) in the first case or the first element of the range in the second case, which makes it either completely useless or just mostly useless.

It's possible to force either the function or projection to ensure that it can only be interpreted one way or the other, but since the algorithm is sufficiently different (see following section), even if such ambiguity is going to be extremely rare (and possible to deal with even if it does arise), we may as well avoid the issue entirely.

As such, this paper proposes a differently named algorithm for the version that takes no initial value rather than adding an overload under the same name.

4.1.2 `optional` or UB?

The result of `ranges::foldl(empty_range, init, f)` is just `init`. That is straightforward. But what would the result of `ranges::foldl1(empty_range, f)` be? There are two options:

1. a disengaged `optional<T>`, or

2. `T`, but this case is undefined behavior

In other words: empty range is either a valid input for the algorithm, whose result is `nullopt`, or there is a precondition that the range is non-empty.

Users can always recover the undefined behavior case if they want, by writing `*foldl1(empty_range, f)`, and the `optional` return allows for easy addition of other functionality, such as providing a sentinel value for the empty range case (`foldl1(empty_range, f).value_or(sentinel)` reads better than `not ranges::empty(r) ? foldl1(r, f) : sentinel`, at least to me). It's also much safer to use in the context where you may not know if the range is empty or not, because it's adapted: `foldl1(r | filter(f), op)`.

However, this would be the very first algorithm in the standard library that meaningful interacts with one of the sum types. And goes against the convention of algorithms simply being undefined for empty ranges (such as `max`). Although it's worth pointing out that `max_element` is *not* UB for an empty range, it simply returns the end iterator, and the distinction there is likely due to simply not having had an available sentinel to return. But now we do.

This paper proposes returning `optional<T>`. Which is added motivation for a name distinct from the `fold` algorithm that takes an initializer.

§ 4.2 `fold_right`

While `ranges::fold` would be a left-fold, there is also occasionally the need for a *right*-fold. As with the previous section, we should also provide overloads of `fold_right` that do not take an initial value.

There are three questions that would need to be asked about `fold_right`.

First, the order of operations of to the function. Given `fold_right([1, 2, 3], z, f)`, is the evaluation `f(f(f(z, 3), 2), 1)` or is the evaluation `f(1, f(2, f(3, z)))`? Note that either way, we're choosing the `3` then `2` then `1`, both are right folds. It's a question of if the initial element is the left-hand operand (as it is in the left `fold`) or the right-hand operand (as it would be if consider the right fold as a flip of the left fold).

One advantage of the former - where the initial call is `f(z, 3)` - is that we can specify `fold_right(r, z, op)` as precisely `fold_left(views::reverse(r), z, op)` and leave it at that. Notably with the same `op`. With the latter - where the initial call is `f(3, z)` - we would need slightly more specification and would want to avoid saying `flip(op)` since directly invoking the operation with the arguments in the correct order is a little better in the case of ranges of prvalues.

If we take a look at how other languages handle left-fold and right-fold, and whether the accumulator is on the same side (and, in these cases, the accumulator is always on the right) or opposite side (the accumulator is on the left-hand side for left fold and on the right-hand side for right fold):

Same Side	Opposite Side
Scheme	Haskell
Elixir	F#

Elm	Julia
	Kotlin
	OCaml
	Scala

This paper chooses what appears to be the more common approach: the accumulator is on the left-hand side for left fold and the right-hand side for right fold. That is, `foldr(r, z, op)` is equivalent to `foldl(reverse(r), z, flip(op))`.

Second, supporting bidirectional ranges is straightforward. Supporting forward ranges involves recursion of the size of the range. Supporting input ranges involves recursion and also copying the whole range first. Are either of these worth supporting? The paper simply supports bidirectional ranges.

Third, the naming question.

§ 4.2.1 Naming for left and right folds

There are roughly four different choices that we could make here:

1. Provide the algorithms `fold` (a left-fold) and `fold_right`.
2. Provide the algorithms `fold_left` and `fold_right`.
3. Provide the algorithms `fold_left` and `fold_right` and also provide an alias `fold` which is also `fold_left`.
4. Provide the algorithms `foldl` and `foldr`.

There's language precedents for any of these cases. F# and Kotlin both provide `fold` as a left-fold and suffixed right-fold (`foldBack` in F#, `foldRight` in Kotlin). Elm, Haskell, Julia, and OCaml provide symmetrically named algorithms (`foldl`/`foldr` for the first three and `fold_left`/`fold_right` for the third). Scala provides a `foldLeft` and `foldRight` while also providing `fold` to also mean `foldLeft`.

In C++, we don't have precedent in the library at this point for providing an alias for an algorithm, although we do have precedent in the library for providing an alias for a range adapter (`keys` and `values` for `elements<0>` and `elements<1>`, and [P2321R0] proposes `pairwise` and `pairwise_transform` as aliases for `adjacent<2>` and `adjacent_transform<2>`). We also have precedent in the library for asymmetric names (`sort` vs `stable_sort` vs `partial_sort`) and symmetric ones (`shift_left` vs `shift_right`), even symmetric ones with terse names (`rotl` and `rotr`, although the latter are basically instructions).

All of which is to say, I don't think there's a clear answer to this question. I think at this point, all possible options have appeared in some revision of this paper. This current revision uses Option 2.

With the versions of algorithms that use an element from the range as the initializer, there's a further choice. LEWG recently voted to prefer `_first` as a suffix to `1`, which is fine for a left fold but with a right fold `_first` seems like a poor choice of suffix because it's really the *last* element that is the initializer. If we drop Option 4 above (LEWG prefers `_left` and `_right` as suffixes to `l` and `r`), we additionally have two options:

1. `fold_right_first`
2. `fold_right_last`

Or, putting it all together, we have the following name banks:

	Left Folds		Right Folds	
	With Init	**No Init**	**With Init**	**No Init**
A	`fold`	`fold_first`	`fold_right`	`fold_right_first`
B	`fold`	`fold_first`	`fold_right`	`fold_right_last`
C	`fold_left`	`fold_left_first`	`fold_right`	`fold_right_first`
D	`fold_left`	`fold_left_first`	`fold_right`	`fold_right_last`
E	`fold ==` `fold_left`	`fold_first ==` `fold_left_first`	`fold_right`	`fold_right_first`
F	`fold ==` `fold_left`	`fold_first ==` `fold_left_first`	`fold_right`	`fold_right_last`

The current revision of this paper proposes Option D: the symmetric names (no `fold` by itself, whether an alias or not) and `_last` as the suffix rather than `_first`. There was previously preference for `fold` over `fold_left`, but this will have to be discussed again separately.

4.3 Short-circuiting folds

The folds discussed up until now have always evaluated the entirety of the range. That's very useful in of itself, and several other algorithms that we have in the standard library can be implemented in terms of such a fold (e.g. `min` or `count_if`).

But for some algorithms, we really want to short circuit. For instance, we don't want to define `all_of(r, pred)` as `fold(r, true, logical_and(), pred)`. This formulation would give the correct answer, but we really don't want to keep evaluating `pred` once we got our first `false`. To do this correctly, we really need short circuiting.

There are (at least) three different approaches for how to have a short-circuiting fold. Here are different approaches to implementing `any_of` in terms of a short-circuiting fold:

1. You could provide a function that mutates the accumulator and returns `true` to continue and `false` to break. That is, `all_of(r, pred)` would look like

```
return fold_while(r, true, [&](bool& state, auto&& elem){
    state = pred(elem);
    return not state;
});
```

and the main loop of the `fold_while` algorithm would look like:

```
for (; first != last; ++first) {
    if (not f(init, *first)) { // NB: not moving init into "f"
        break;                 // since we're mutating in-place
```

```
    }
}
return init;
```

2. Same as #1, except return an enum class instead of a `bool` (like `control_flow::continue_` vs `control_flow::break_`).

3. You could provide a function that returns a `variant<continue_<T>, break_<U>>`. The algorithm would then return this variant (rather than a `T`). Rust's `Itertools` crate provides this under the name `fold_while`:

```
template <typename T> struct continue_ { T value; };

template <typename T> struct break_ { T value; };
template <> struct break_<void> { };
break_() -> break_<void>;

template <typename T, typename U>
using fold_while_t = variant<continue_<T>, break_<U>>;

return fold_while(r, true, [&](bool, auto&& elem) -> fold_while_t<bool, void> {
    if (pred(elem)) {
        return continue_{true};
    } else {
        return break_();
    }
}).index() == 0;
```

and the main loop of the `fold_while` algorithm would look like:

```
variant<continue_<T>, break_<E>> state = continue_<T>{init};

for (; first != last; ++first) {
    state = f(get<continue_<T>>(move(state)).value, *first);
    if (holds_alternative<break_<U>>(next_state)) {
        break;
    }
}
return state;
```

4. You could provide a function that returns an `expected<T, E>`, which then the algorithm would return an `expected<T, E>` (rather than a `T`). Rust `Iterator` trait provides this under the name `try_fold`:

```
return fold_while(r, true, [&](bool, auto&& elem) -> expected<bool, bool> {
    if (pred(FWD(elem))) {
        return true;
    } else {
        return unexpected(false);
    }
}).has_value();
```

and the main loop of the `fold_while` algorithm would look like:

```
for (; first != last; ++first) {
    auto next = f(move(init), *first);
    if (not next) {
        return next;
    }
    init = *move(next);
}

return init;
```

Option (1) is a questionable option because of mutating state (note that we cannot use `predicate` as the constraint on the type, because `predicates` are not allowed to mutate their arguments), but this approach is probably the most efficient due to not moving the accumulator at all.

Option (2) seems like a strictly worse version than Option (1) for C++, due to not being able to use existing predicates.

Option (3) is an awkward option for C++ because of general ergonomics. The provided lambda couldn't just return `continue_{x}` in one case and `break_{y}` in another since those have different types, so you'd basically always have to provide `-> fold_while_t<T, U>` as a trailing-return-type. This would also be the first (or second, see above) algorithm which actually meaningfully uses one of the standard library's sum types.

Option (4) isn't a great option for C++ because we don't even have `expected<T, E>` in the standard library yet (although hopefully imminent at this point, as [P0323R10] is already being discussed in LWG), and ergonomically it has the same issues as described earlier - you can't just return a `T` and an `unexpected<E>` for a lambda, you need to have a trailing return type. We'd also want to generalize this approach to any "truthy" type which would require coming up with a way to conceptualize (in the `concept` sense) "truthy" (since `optional<T>` would be a valid type as well, as well as any other the various user-defined versions out there. Rust's name for this is `Try`, with a new revision in progress). Because the implementation would have to wrap and unwrap the accumulator, it could potentially be less efficient than Option (1) as well.

Or, to put these all in a table, given that the accumulator has type `T`:

	Callable is Pure?	Callable returns…	Algorithm returns…
1	❌	`bool`	`T`
2	❌	`enum class control_flow`	`T`
3	✔️	`variant<continue_<T>, break_<U>>`	`variant<continue_<T>, break_<U>>`
4	✔️	`expected<T, E>` `optional<T>`	`expected<T, E>` `optional<T>`

Option (3) is far too unergonomic in C++ to be reasonable, but Option (4) does have the benefit that it would allow different return types for the early-return and full-consume cases.

None of these options stand out as particularly promising. Moreover, I've been unable to find any other other than Rust which provides such an algorithm (even though basically every language provides algorithms that reduce to a short-circuiting fold, like `any` and `all`), and in Rust, this algorithm is quite ergonomic due to language features that C++ does not possess. As such, while a previous revision of this paper proposed a short-circuiting fold (specifically, Option 1), this paper does not.

4.4 Iterator-returning folds

Up until this point, this paper has only discussed returning a *value* from `fold`: whatever we get as the result of `f(f(f(f(init, e0), e1), e2), e3)`. But there is another value that we compute along the way that is thrown out: the end iterator.

An alternative formulation of `fold` would preserve that information. Rather than returning `R`, we could do something like this:

```
template <input_iterator I, typename R>
struct fold_result {
    I in;
    R value;
};

template <input_iterator I, sentinel_for<I> S, class T, class Proj = identity,
    indirectly-binary-left-foldable<T, projected<I, Proj>> F,
    typename R = invoke_result_t<F&, T, indirect_result_t<Proj&, I>>>
constexpr auto fold_left(I first, S last, T init, F f, Proj proj = {})
    -> fold_result<I, R>;
```

But the problem with that direction is, quoting from [P2214R0]:

> [T]he above definition definitely follows Alexander Stepanov's law of useful return [stepanov] (emphasis ours):
>
> > When writing code, it's often the case that you end up computing a value that the calling function doesn't currently need. Later, however, this value may be important when the code is called in a different situation. In this situation, you should obey the law of useful return: *A procedure should return all the potentially useful information it computed.*
>
> But it makes the usage of the algorithm quite cumbersome. The point of a fold is to return the single value. We would just want to write:
>
> ```
> int total = ranges::sum(numbers);
> ```
>
> Rather than:
>
> ```
> auto [_, total] = ranges::sum(numbers);
> ```
>
> or:
>
> ```
> int total = ranges::sum(numbers, 0).value;
> ```

> `ranges::fold` should just return `T`. This would be consistent with what the other range-based folds already return in C++20 (e.g. `ranges::count` returns a `range_difference_t<R>`, `ranges::any_of` - which can't quite be a `fold` due to wanting to short-circuit - just returns `bool`).

Moreover, even if we added a version of this algorithm that returned an iterator (let's call it `fold_with_iterator`), we wouldn't want `fold(first, last, init, f)` to be defined as

```
return fold_with_iterator(first, last, init, f).value;
```

since this would have to incur an extra move of the accumulated result, due to lack of copy elision (we have different return types). So we'd want need this algorithm to be specified separately (or, perhaps, the "*Effects*: equivalent to" formulation is sufficiently permissive as to allow implementations to do the right thing?)

From a usability perspective, I think it's important that `fold` just return the value.

The problem going past that is that we end up with this combinatorial explosion of algorithms based on a lot of orthogonal choices:

1. iterator pair or range
2. left or right fold
3. initial value or no initial value
4. short-circuit or not short-circuit
5. return `T` or `(iterator, T)`

Which would be… 32 distinct functions (under 16 different names) if we go all out. And these really are basically orthogonal choices. Indeed, a short-circuiting fold seems even more likely to want the iterator that the algorithm stopped at! Do we need to provide all of them? Maybe we do!

This brings with it its own naming problem. That's a lot of names. One approach there could be a suffix system:

— `fold_left` is a non-short-circuiting left-fold with an initial value that returns `T`

— `fold_left_first` is a non-short-circuiting left-fold with no initial value that returns `T`

— `fold_left_first_while` is a short-circuiting left-fold with no initial value that returns `T`

— `fold_right_with_iter` is a non-short-circuiting right-fold with an initial value that returns `(iterator, T)`

— `fold_right_last_while_with_iter` is a short-circuiting right-fold with no initial value that returns `(iterator, T)`

`with_iter` is not the best suffix, but the rest seem to work out ok.

§ 4.5 The proposal: iterator-returning left folds

One solution to the combinatorial explosion problem, as suggested by Tristan Brindle, is to simply not consider all of these options as being necessarily orthogonal. That is, providing a left-fold that returns a value is very valuable and highly desired. But having both a left- and right-fold that return an iterator?

In other words, if you want the common and convenient thing, we'll provide that: `foldl` and `foldl1` will exist and just return a value. But if you want a more complex tool, it'll be a little more complicated to use. In other words, what this paper is proposing is six algorithms (with two overloads each):

1. `fold_left` (a left-fold with an initial value that returns `T`)
2. `fold_left_first` (a left-fold with no initial value that returns `T`)
3. `fold_right` (a right-fold with an initial value that returns `T`)
4. `fold_right_last` (a right-fold with no initial value that returns `T`)
5. `fold_left_with_iter` (a left-fold with an initial value that returns `(iterator, T)`)
6. `fold_left_first_with_iter` (a left-fold with no initial value that returns `(iterator, T)`)

There is no right-fold that returns an iterator, since you can use `views::reverse`. This is a little harder to use since the transition from a fold that just returns `T` to a fold that actually produces an `iterator` as well isn't as easy as going from:

```
auto value = ranges::fold_left(r, init, op);
```

to:

```
auto [iterator, value] = ranges::fold_left_with_iter(r, init, op);
```

Instead, you have to flip the binary operation - and we have no `flip` function adapter (Boost.HOF does though).

§ 4.6 No projections

All the algorithms in `std::ranges` take a projection, but there's a problem in this case. For `fold_left`, there's no problem - we just look over the iterators and do `invoke(proj, *first)` before passing that into the call to `f`.

But for `fold_left_first`, we need to produce an initial *value*, rather than an initial *reference*. And with a projection, we can't… do that for iterators that give proxy references. Let's start by writing out what `fold_left_first` would be implemented as, without a projection, and then try to add it in:

```
template <input_iterator I, sentinel_for<I> S,
  indirectly-binary-left-foldable<iter_value_t<I>, I> F>
  requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
constexpr auto fold_left_first(I first, S last, F f) {
  using U = decltype(ranges::fold_left(first, last, iter_value_t<I>(*first),
        f));
  if (first == last) {
    return optional<U>(nullopt);
  }
```

```
  iter_value_t<I> init(*first); // <==
  ++first;
  return optional<U>(in_place,
    ranges::fold_left(std::move(first), std::move(last), std::move(init),
        std::move(f)));
}
```

Now, how do we replace the commented line with a projected version of the same? We need a *value*. Imagine if `*first` yielded a `tuple<T&>` (as it does for `zip`) and our projection is `identity` (the common case). If we did:

```
iter_value_t<projected<I, Proj>> init(invoke(proj, *first));
```

We'd still end up with a `tuple<T&>`, whereas we'd need to end up with a `tuple<T>`. The only way to get this right is to project the value type first:

```
auto init = invoke(proj, iter_value_t<I>(*first));
```

But that's not actually required to be valid. The projection is required to take either `iter_reference_t<I>` or `iter_value_t<I>&` (specifically an *lvalue* of the value type). So you *have* to write this:

```
iter_value_t<I> init_(*first);
auto init = invoke(proj, init_);
```

Which means that the only way to get this right is to incur an extra copy.

Which means we're performing an extra copy in the typical case (the iterator does not yield a proxy reference and there is no projection) simply to be able to support the rare case. That seems problematic. And we don't have any other algorithm where we'd need to get a projected *value*, we only ever need projected *references* (algorithms like `min` which return a value type return the value type of the original range, not the projected range).

This suggests dropping the projections for the `*_first`/`*_last` variants (`fold_left_first`, `fold_right_last`, and `fold_left_first_with_iter`). But then they're more inconsistent with the other variants, so this paper suggests simply not having projections at all. This doesn't cost you anything for some of the algorithms, since projections in these case can always be provided either as range adaptors or function adaptors.

The expression that I'm not supporting:

```
fold_left(r, init, f, proj)
```

means the same as either this version using a range adaptor:

```
fold_left(r | views::transform(proj), init, f)
```

or this version using a function adaptor. Boost.HOF defines an adaptor `proj` such that `proj(p, f)(xs...)` means `f(p(xs)...)`. We need to project only the right hand argument, so here `proj_rhs(p, f)`

`(x, y)` means `f(x, p(y))` as desired. One way to implement this is `std::bind(f, _1, std::bind(proj, _2))`.

```
fold_left(r, init, proj_rhs(proj, f))
```

The version using `views::transform` is probably more familiar, but it won't work as well for `fold_left_with_iter` since this would return `dangling` (since `transform` is never a borrowed range) but even with adjustment would return an iterator into the transformed range rather than the original range. This issue is one of the original cited motivations for projections in [N4128]. The version using the function adaptor has no such issue, although proposing more function adaptors into the standard library is out of scope for this paper.

In short, there are good reasons to avoid projections for `fold_left_first`, `fold_right_last`, and `fold_left_first_with_iter`. For consistency, I'm also removing the projections for `fold_left`, `fold_right`, and `fold_left_with_iter`. This makes the folds inconsistent with the rest of the standard library algorithms (although at least internally consistent), but it doesn't actually cost them functionality.

§ 5 Implementation Experience

Part of this paper (naming the algorithms `foldl`, `foldl1`, `foldr`, and `foldr1`, where the `*1` algorithms return an `optional`) has been implemented in range-v3 [range-v3-fold] (with projections, although not quite correctly as it turns out).

The algorithms returning an iterator have not been implemented in range-v3 yet, but they're exactly the same as their non-iterator-returning counterparts.

§ 6 Wording

Add a feature-test macro to 17.3.2 [version.syn]:

```
#define __cpp_lib_fold 2021XXL // also in <algorithm>
```

Append to 25.4 [algorithm.syn], first a new result type:

```
#include <initializer_list>

namespace std {
  namespace ranges {
    // [algorithms.results], algorithm result types
    template<class I, class F>
      struct in_fun_result;

    template<class I1, class I2>
      struct in_in_result;

    template<class I, class O>
      struct in_out_result;

    template<class I1, class I2, class O>
```

```
      struct in_in_out_result;

    template<class I, class O1, class O2>
      struct in_out_out_result;

    template<class T>
      struct min_max_result;

    template<class I>
      struct in_found_result;

+   template<class I, class T>
+     struct in_value_result;
  }

  // ...
}
```

and then also a bunch of fold algorithms:

```
namespace std {
  // ...

  // [alg.fold], folds
  namespace ranges {
    template<class F>
    struct flipped {  // exposition only
      F f;

      template<class T, class U>
        requires invocable<F&, U, T>
      invoke_result_t<F&, U, T> operator()(T&&, U&&);
    };

    template <class F, class T, class I, class U>
    concept indirectly-binary-left-foldable-impl =  // exposition only
        movable<T> &&
        movable<U> &&
        convertible_to<T, U> &&
        invocable<F&, U, iter_reference_t<I>> &&
        assignable_from<U&, invoke_result_t<F&, U, iter_reference_t<I>>>;

    template <class F, class T, class I>
    concept indirectly-binary-left-foldable =      // exposition only
        copy_constructible<F> &&
        indirectly_readable<I> &&
        invocable<F&, T, iter_reference_t<I>> &&
        convertible_to<invoke_result_t<F&, T, iter_reference_t<I>>,
               decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>> &&
        indirectly-binary-left-foldable-impl<F, T, I,
         decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>>;

    template <class F, class T, class I>
    concept indirectly-binary-right-foldable =    // exposition only
        indirectly-binary-left-foldable<flipped<F>, T, I>;
```

```
template<input_iterator I, sentinel_for<I> S, class T,
  indirectly-binary-left-foldable<T, I> F>
constexpr auto fold_left(I first, S last, T init, F f);

template<input_range R, class T,
  indirectly-binary-left-foldable<T, iterator_t<R>> F>
constexpr auto fold_left(R&& r, T init, F f);

template <input_iterator I, sentinel_for<I> S,
  indirectly-binary-left-foldable<iter_value_t<I>, I> F>
  requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
constexpr auto fold_left_first(I first, S last, F f);

template <input_range R,
  indirectly-binary-left-foldable<range_value_t<R>, iterator_t<R>> F>
  requires constructible_from<range_value_t<R>, range_reference_t<R>>
constexpr auto fold_left_first(R&& r, F f);

template<bidirectional_iterator I, sentinel_for<I> S, class T,
  indirectly-binary-right-foldable<T, I> F>
constexpr auto fold_right(I first, S last, T init, F f);

template<bidirectional_range R, class T,
  indirectly-binary-right-foldable<T, iterator_t<R>> F>
constexpr auto fold_right(R&& r, T init, F f);

template <bidirectional_iterator I, sentinel_for<I> S,
  indirectly-binary-right-foldable<iter_value_t<I>, I> F>
  requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
constexpr auto fold_right_last(I first, S last, F f);

template <bidirectional_range R,
  indirectly-binary-right-foldable<range_value_t<R>, iterator_t<R>> F>
  requires constructible_from<range_value_t<R>, range_reference_t<R>>
constexpr auto fold_right_last(R&& r, F f);

template<class I, class T>
  using fold_left_with_iter_result = in_value_result<I, T>;
template<class I, class T>
  using fold_left_first_with_iter_result = in_value_result<I, T>;

template<input_iterator I, sentinel_for<I> S, class T,
   indirectly-binary-left-foldable<T, I> F>
constexpr see below fold_left_with_iter(I first, S last, T init, F f);

template<input_range R, class T,
  indirectly-binary-left-foldable<T, iterator_t<R>> F>
constexpr see below fold_left_with_iter(R&& r, T init, F f);

template <input_iterator I, sentinel_for<I> S,
  indirectly-binary-left-foldable<iter_value_t<I>, I> F>
  requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
constexpr see below fold_left_first_with_iter(I first, S last, F f);

template <input_range R,
```

```
      indirectly-binary-left-foldable<range_value_t<R>, iterator_t<R>> F>
      requires constructible_from<range_value_t<R>, range_reference_t<R>>
    constexpr see below fold_left_first_with_iter(R&& r, F f);
  }
}
```

Add a new result type to 25.5 [algorithms.results]:

```
namespace std::ranges {
  // ...

  template<class I>
  struct in_found_result {
    [[no_unique_address]] I in;
    bool found;

    template<class I2>
      requires convertible_to<const I&, I2>
    constexpr operator in_found_result<I2>() const & {
      return {in, found};
    }
    template<class I2>
      requires convertible_to<I, I2>
    constexpr operator in_found_result<I2>() && {
      return {std::move(in), found};
    }
  };

+ template<class I, class T>
+ struct in_value_result {
+   [[no_unique_address]] I in;
+   [[no_unique_address]] T value;
+
+   template<class I2, class T2>
+     requires convertible_to<const I&, I2> && convertible_to<const T&, T2>
+   constexpr operator in_value_result<I2, T2>() const & {
+     return {in, value};
+   }
+
+   template<class I2, class T2>
+     requires convertible_to<I, I2> && convertible_to<T, T2>
+   constexpr operator in_value_result<I2, T2>() && {
+     return {std::move(in), std::move(value)};
+   }
+ };
}
```

And add a new clause, [alg.fold]:

```
template<input_iterator I, sentinel_for<I> S, class T,
  indirectly-binary-left-foldable<T, I> F>
constexpr auto ranges::fold_left(I first, S last, T init, F f);

template<input_range R, class T,
```

```
  indirectly-binary-left-foldable<T, iterator_t<R>> F>
constexpr auto ranges::fold_left(R&& r, T init, F f);
```

1 *Returns*: `ranges::fold_left_with_iter(std::move(first), last, std::move(init), f).value`.

```
template <input_iterator I, sentinel_for<I> S,
  indirectly-binary-left-foldable<iter_value_t<I>, I> F>
  requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
constexpr auto ranges::fold_left_first(I first, S last, F f);

template <input_range R,
  indirectly-binary-left-foldable<range_value_t<R>, iterator_t<R>> F>
  requires constructible_from<range_value_t<R>, range_reference_t<R>>
constexpr auto ranges::fold_left_first(R&& r, F f);
```

2 *Returns*: `ranges::fold_left_first_with_iter(std::move(first), last, f).value`.

```
template<bidirectional_iterator I, sentinel_for<I> S, class T,
  indirectly-binary-right-foldable<T, I> F>
constexpr auto ranges::fold_right(I first, S last, T init, F f);

template<bidirectional_range R, class T,
  indirectly-binary-right-foldable<T, iterator_t<R>> F>
constexpr auto ranges::fold_right(R&& r, T init, F f);
```

3 *Effects*: Equivalent to:

```
using U = decay_t<invoke_result_t<F&, iter_reference_t<I>, T>>;

if (first == last) {
    return U(std::move(init));
}

I tail = ranges::next(first, last);
U accum = invoke(f, *--tail, std::move(init));
while (first != tail) {
    accum = invoke(f, *--tail, std::move(accum));
}
return accum;
```

```
template <bidirectional_iterator I, sentinel_for<I> S,
  indirectly-binary-right-foldable<iter_value_t<I>, I> F>
  requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
constexpr auto ranges::fold_right_last(I first, S last, F f);

template <bidirectional_range R,
  indirectly-binary-right-foldable<range_value_t<R>, iterator_t<R>> F>
  requires constructible_from<range_value_t<R>, range_reference_t<R>>
constexpr auto ranges::fold_right_last(R&& r, F f);
```

4 Let `U` be `decltype(ranges::fold_right(first, last, iter_value_t<I>(*first), f))`.

5 *Effects*: Equivalent to:

```
    if (first == last) {
        return optional<U>();
    }

    I tail = ranges::prev(ranges::next(first, std::move(last)));
    return optional<U>(in_place,
        ranges::fold_right(std::move(first), tail, iter_value_t<I>
            (*tail), std::move(f)));
```

```
template<input_iterator I, sentinel_for<I> S, class T,
  indirectly-binary-left-foldable<T, I> F>
constexpr see below fold_left_with_iter(I first, S last, T init, F f);

template<input_range R, class T,
  indirectly-binary-left-foldable<T, iterator_t<R>> F>
constexpr see below fold_left_with_iter(R&& r, T init, F f);
```

6 Let `U` be `decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>`.

7 *Effects*: Equivalent to:

```
    if (first == last) {
        return {std::move(first), U(std::move(init))};
    }

    U accum = invoke(f, std::move(init), *first);
    for (++first; first != last; ++first) {
        accum = invoke(f, std::move(accum), *first);
    }
    return {std::move(first), std::move(accum)};
```

8 *Remarks*: The return type is `fold_left_with_iter_result<I, U>` for the first overload and `fold_left_with_iter_result<borrowed_iterator_t<R>, U>` for the second overload.

```
template <input_iterator I, sentinel_for<I> S,
  indirectly-binary-left-foldable<iter_value_t<I>, I> F>
  requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
constexpr see below fold_left_first_with_iter(I first, last, F f)

template <input_range R,
  indirectly-binary-left-foldable<range_value_t<R>, iterator_t<R>> F>
  requires constructible_from<range_value_t<R>, range_reference_t<R>>
constexpr see below fold_left_first_with_iter(R&& r, F f);
```

9 Let `U` be `decltype(ranges::fold_left(std::move(first), last, iter_value_t<I>(*first), f))`.

10 *Effects*: Equivalent to:

```
    if (first == last) {
        return {std::move(first), optional<U>()};
    }

    optional<U> init(in_place, *first);
    for (++first; first != last; ++first) {
```

```
        *init = invoke(f, std::move(*init), *first);
    }
    return {std::move(first), std::move(init)};
```

11 *Remarks*: The return type is `fold_left_first_with_iter_result<I, optional<U>>` for the first overload and `fold_left_first_with_iter_result<borrowed_iterator_t<R>, optional<U>>` for the second overload.

§ 7 References

[iterator_fold_self] Ashley Mannix. 2020. Tracking issue for `iterator_fold_self`.
https://github.com/rust-lang/rust/issues/68125

[N4128] E. Niebler, S. Parent, A. Sutton. 2014-10-10. Ranges for the Standard Library, Revision 1.
https://wg21.link/n4128

[P0323R10] JF Bastien, Vicente Botet. 2021-04-15. std::expected.
https://wg21.link/p0323r10

[P2214R0] Barry Revzin, Conor Hoekstra, Tim Song. 2020-10-15. A Plan for C++23 Ranges.
https://wg21.link/p2214r0

[P2321R0] Tim Song. 2021-02-21. zip.
https://wg21.link/p2321r0

[P2322-minutes] LEWG. 2021. P2322 Minutes.
https://wiki.edg.com/bin/view/Wg21telecons2021/P2322#2021-05-03

[P2322R0] Barry Revzin. 2021-02-18. ranges::fold.
https://wg21.link/p2322r0

[P2322R1] Barry Revzin. 2021-03-17. ranges::fold.
https://wg21.link/p2322r1

[P2322R2] Barry Revzin. 2021-04-15. ranges::fold.
https://wg21.link/p2322r2

[P2322R3] Barry Revzin. 2021-06-13. ranges::fold.
https://wg21.link/p2322r3

[P2322R4] Barry Revzin. 2021-09-13. ranges::fold.
https://wg21.link/p2322r4

[range-v3-fold] Barry Revzin. 2021. Fold algos.
https://github.com/ericniebler/range-v3/pull/1628

[stepanov] Alexander A. Stepanov. 2014. From Mathematics to Generic Programming.